

Seven Seas A-52

Security Audit

July 15, 2025

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Security Levels Reference](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security assessment of Kinetiq was performed by the Macro security team on July 10th 2025.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

No issues or discrepancies were identified within the defined scope of this audit.

Specification

Our understanding of the specification was based on the following sources:

Security Assessment:

Kinetiq

- **General**

- Documentation: No available documentation
- A liquid staking protocol allowing users to stake the HyperEVM's native HYPE token to be used by trusted validators to stake tokens and distribute yield to KHYPE holders.
- Main components are:
 - Staking Manager: Contract where users stake their HYPE and receive KHYPE, and manages validators as well as executing L1 staking operations.
 - KHYPE: The erc20 token users receive on staking
 - Validator Manager: Manages validators, can activate/deactivate validators as well as handling validator performance and slashing events.
 - Staking Accountant: Keeps track of stacked amount and claims, and uses validator manager values to determine exchange rate of Hype to KHYPE.
 - Oracle Manager: Keeps track of oracles used to calculate a validators performance.

- **Security Evaluation**

- [Audited by Zenith, Spearbit, Pashov, and Code4Arena](#)

All contracts reviewed except the `DefaultOracle` are deployed as a [TransparentUpgradeableProxy](#) each with the same [EOA address](#) as the admin, which has upgrade privileges. This leaves a singular point of failure for the entire protocol. It is better practice to use a multi-sig timelocked wallet for important permissions like contract upgrades.

These contracts require a permissioned functions to be called and uses [AccessControl](#) to manage different permissioned roles. This is good practice, however the admin and each specific role throughout the protocol is all the same [EOA address](#) as of this review. It is expected to have these roles be distributed across multiple secure addresses. This may be the case for now as it is yet to be publicly launched, and roles could be distributed in the near future.

It is notable that most of these contracts are able to be paused, including the KHYPE token and staking manager, and withdrawals from the staking manager can be prevented by a permissioned address, but these are standard and good practice provided the trusted addresses are acting in the best interest of the protocol.

These contracts have been sufficiently reviewed by many notable auditing firms, and have resolved the issues presented, giving confidence that there are no lingering exploits and the contract will operate as expected.

Rating: C (Single point of failure, lack of role distribution, early stages of protocol)

Suggestion: Use a more secure proxy admin owner and diversify protocol roles to multiple trusted and secure addresses, ideally using a timelock where able.

Source Code

The following source code was reviewed during the audit:

Specifically, we conducted a security review based the following Kinetiq contracts:

- [PauserRegistry](#): `0x752E76ea71960Da08644614E626c9F9Ff5a50547`
 - [Implementation](#): `0x5e6D0eAc0ef26F49cbE2B772ED897330389f87E7`
- [StakingManager](#): `0x393D0B87Ed38fc779FD9611144aE649BA6082109`
 - [Implementation](#): `0x9eC8bA489327120908B8E6c9Cdf6214770cd6984`
- [KHYPE](#): `0xfD739d4e423301CE9385c1fb8850539D657C296D`
 - [Implementation](#): `0x210461FcA55b84F4F03a19fF3DFAe6d0FfAE3675`
- [ValidatorManager](#): `0x4b797A93DfC3D18Cf98B7322a2b142FA8007508f`
 - [Implementation](#): `0x4b797A93DfC3D18Cf98B7322a2b142FA8007508f`
- [StakingAccountant](#): `0x9209648Ec9D448EF57116B73A2f081835643dc7A`
 - [Implementation](#): `0xecaa7Cd734668E086812Bd4241D6FC0260DBa513`
- [OracleManager](#): `0x192826e470bd65FDC2CB472eDd834D096233049b`
 - [Implementation](#): `0x96b64DCfBf93eeAD60E3d0474334Fc081161c814`
- [DefaultOracle](#): `0xefbcCc6E33DA1C1ef638cBc0F044968D0f590fED`

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Descriptions and Recommendations

There were no significant issues found in the code reviewed.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.